

Advanced Backend Development – JavaScript & Node.js Deep Dive

Notes for MCQ-based exams

Covers definitions, code examples, comparisons, common pitfalls, and "why this is better" facts.

Each section is structured with subtopics, practical code, and MCQ-oriented highlights.

1. JavaScript Runtime

1.1 What is a Runtime?

- A **runtime** provides the environment needed to execute JavaScript code.
- In the browser, the runtime includes the JavaScript engine (e.g., V8), Web APIs (DOM, timers, fetch), callback queue, and the event loop.
- On the server, **Node.js** provides the runtime (V8 + libuv for async I/O, timers, file system, etc.).

1.2 Key Components

Component	Browser	Node.js
JS Engine	V8 (Chrome), SpiderMonkey	V8
Event Loop	Yes (via browser)	Yes (libuv)
Web/Node APIs	DOM, Fetch, setTimeout	fs, http, crypto, process
Microtask Queue	Promise callbacks	Same (Promise, nextTick)
Callback Queue	setTimeout, setInterval	Timers, I/O callbacks

MCQ Fact:

The JavaScript engine is single-threaded. Asynchronous operations are handled by the runtime environment (Web APIs / libuv), not by the engine itself.

1.3 Execution Model

[Call Stack] → [Web APIs / Node APIs] → [Task Queues] → [Event Loop]

- **Call Stack:** synchronous execution, LIFO.

- **Event Loop:** picks tasks from the callback queue only when the call stack is empty.
- **Microtask Queue:** processed after each task, before the next task.

Common MCQ:

What will happen if you block the call stack?

The event loop cannot pick up new tasks; the application freezes (unresponsive).

2. Execution Context

2.1 Definition

An **execution context** is an abstract concept that holds information about the environment within which the current code is being executed. It consists of:

- **Variable Environment** (let, const, var bindings)
- **Lexical Environment** (identifier resolution based on where code is written)
- **this binding**

2.2 Types of Execution Contexts

1. **Global Execution Context (GEC)** – created when the script first runs.
2. **Function Execution Context (FEC)** – created each time a function is called.
3. **Eval Execution Context** – code executed inside `eval()`.

2.3 Creation Phase (Hoisting)

1. Create the **Variable Environment**:
 - var declarations are hoisted and initialised with undefined.
 - let/const are hoisted but not initialised (Temporal Dead Zone).
 - Function declarations are hoisted entirely.
2. Set the `this` value.
3. Establish the **scope chain**.

MCQ Fact:

A `let` variable is in the Temporal Dead Zone (TDZ) from the start of the block until the declaration is evaluated. Accessing it before the declaration causes a `ReferenceError`. A `var` returns `undefined`.

```
console.log(a); // undefined (var hoisted)
var a = 10;
```

```
console.log(b); // ReferenceError: Cannot access 'b' before initializ
let b = 20;
```

2.4 Execution Phase

- Code runs line by line.
- Variables get assigned.
- Function invocations create new FECs and push onto the call stack.

Diagram of a function call:

Call Stack:

1. GEC
2. FEC of outerFunction
3. FEC of innerFunction

After innerFunction returns, its FEC is popped off.

3. Variables (let, const)

3.1 var vs let vs const

Feature	var	let	const
Scope	Function/global	Block	Block
Hoisting	Hoisted (undefined)	Hoisted but TDZ	Hoisted but TDZ
Re-declaration (same scope)	Allowed	Not allowed	Not allowed
Re-assignment	Allowed	Allowed	Not allowed
Initialization required	No	No	Yes

MCQ Trap:

const prevents reassignment of the variable binding, but if it holds an object, the object's properties can be mutated.

```
const obj = { name: 'Alice' };  
obj.name = 'Bob'; // allowed  
obj = { name: 'Charlie' }; // TypeError: Assignment to constant variable
```

3.2 Block Scope and let/const

```

{
  var x = 1;    // function/global scope
  let y = 2;    // block scope
  const z = 3;
}
console.log(x); // 1
console.log(y); // ReferenceError

```

3.3 Temporal Dead Zone (TDZ) in Loops

```

for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i)); // 0, 1, 2 (new binding each iteration)
}

for (var j = 0; j < 3; j++) {
  setTimeout(() => console.log(j)); // 3, 3, 3 (same variable)
}

```

MCQ Fact:

let in loop headers creates a new lexical scope per iteration, preserving the correct value in closures. var does not.

4. Data Types

4.1 Primitive Types (7)

- string, number, bigint, boolean, undefined, symbol, null
- Immutable, compared by value.
- typeof null returns "object" (historical bug).

4.2 Reference Types

- Object, Array, Function, Date, etc.
- Mutable, compared by reference.

```

let a = [1,2,3];
let b = a;
b.push(4);
console.log(a); // [1,2,3,4] (same reference)

```

MCQ Fact:

typeof [] returns "object". To check for array, use Array.isArray().

4.3 Type Coercion – Common MCQs

```
console.log(1 + "2");      // "12" (number + string = string concat)
console.log("5" - 3);      // 2 (string converted to number)
console.log(true + false); // 1 + 0 = 1
console.log(null + 5);     // 0 + 5 = 5 (null -> 0)
console.log(undefined + 5); // NaN (undefined -> NaN)
```

Equality traps:

```
console.log(0 == false);   // true
console.log(0 === false);  // false
console.log(null == undefined); // true
console.log(null === undefined); // false
```

Always use === to avoid type coercion.

5. Functions

5.1 Function Declarations vs Expressions

- **Declaration:** `function name() {}` – hoisted entirely.
- **Expression:** `const name = function() {};` – variable hoisting (TDZ if `let/const`), not function body.

```
foo(); // works
function foo() { console.log('declaration'); }
```



```
bar(); // TypeError: bar is not a function
var bar = function() { console.log('expression'); };
```

5.2 Parameters and Arguments

- Primitive parameters are passed by value; objects by reference (value of the reference).
- `arguments` object (array-like) available in non-arrow functions.

5.3 Default Parameters

```
function multiply(a, b = 2) {
  return a * b;
}
multiply(5); // 10
multiply(5, undefined); // 10 (default kicks in)
multiply(5, null); // 0 (null is not replaced)
```

5.4 this in Regular Functions

- Depends on **how the function is called**, not where it's defined.
- **Global context**: window (browser) or global (Node) / undefined in strict mode.
- **Method call**: the object before the dot.
- **Constructor**: new instance.
- **call/apply/bind**: explicitly set.

MCQ Fact:

In a regular function inside a method, **this** refers to the global object (or undefined in strict mode), not the outer method's **this**. Arrow functions solve this.

6. Arrow Functions

6.1 Syntax

```
const add = (a, b) => a + b;
const square = x => x * x;
const getObj = () => ({ name: 'John' }); // return object literal: { name: 'John' }
```

6.2 Differences from Regular Functions

Feature	Regular Function	Arrow Function
this binding	Dynamic (caller)	Lexical (enclosing scope)
arguments object	Available	Not available
Constructor (new)	Yes	No (no [[Construct]])
prototype property	Yes	No
super	Based on caller	Lexical
Implicit return	No (unless using expression)	Yes with concise body

6.3 Lexical this

```
const obj = {
  name: 'Alice',
  regularFunc: function() {
    setTimeout(function() {
      console.log(this.name); // undefined (global/strict)
    }, 100);
  },
  arrowFunc: function() {
    setTimeout(() => {
      console.log(this.name); // Alice (lexical `this` from ai
    }, 100);
  }
};
obj.regularFunc();
obj.arrowFunc();
```

MCQ Fact:

Arrow functions **cannot be used as methods** that need dynamic this. Using an arrow function as an object method will capture this from the surrounding scope (often global/undefined), leading to unexpected behavior.

```
const obj = {
  name: 'Bob',
  method: () => {
    console.log(this.name); // undefined (lexical `this` likely
  }
};
```

6.4 No arguments object

```
const arrow = () => {
  console.log(arguments); // ReferenceError: arguments is not def
};
// Use rest parameters instead
const arrow2 = (...args) => console.log(args);
```

7. Scope

7.1 Global Scope

- Variables declared outside any function/block (or with var in the global space).

- Accessible everywhere.

7.2 Function Scope

- Variables declared with `var` are scoped to the nearest function.
- `let/const` also respect function scope if declared there.

7.3 Block Scope

- `{ }` defines a block. `let`, `const`, and `class` are block-scoped. `var` ignores block scope.

```
function test() {  
  var x = 1;  
  if (true) {  
    var x = 2; // same variable (function scoped)  
    let y = 3; // different block variable  
  }  
  console.log(x); // 2  
  console.log(y); // ReferenceError  
}
```

7.4 Scope Chain / Lexical Scoping

- Inner functions can access variables of their outer functions.
- Resolves from inner scope → outer scope → global.

```
function outer() {  
  const a = 1;  
  function inner() {  
    const b = 2;  
    console.log(a + b); // 3 (accesses `a` from outer)  
  }  
  inner();  
}
```

MCQ Fact:

Lexical scoping means scope is determined at **compile time** (where the function is defined), not at call time.

8. Closures

8.1 Definition

A **closure** is created when a function retains access to its lexical scope even when the function is executed outside that scope. It combines the function and its surrounding state.

8.2 Basic Example

```
function createCounter() {  
  let count = 0; // private variable  
  return function() {  
    count++;  
    return count;  
  };  
}  
const counter = createCounter();  
console.log(counter()); // 1  
console.log(counter()); // 2
```

count is not accessible directly, only via the returned function. This is data privacy through closures.

8.3 Closure in Loops – Classic MCQ

```
for (var i = 0; i < 3; i++) {  
  setTimeout(function() {  
    console.log(i); // 3, 3, 3  
  }, 0);  
}
```

Explanation: The callback function closes over the same `i` (function scope). By the time it executes, `i` is 3.

Fix with an IIFE (Immediately Invoked Function Expression) or `let`:

```
// IIFE fix  
for (var i = 0; i < 3; i++) {  
  (function(j) {  
    setTimeout(() => console.log(j), 0); // 0,1,2  
  })(i);  
}  
// let fix (block scope)  
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0); // 0,1,2  
}
```

MCQ Fact:

Each iteration of a `let` loop creates a new scope with its own binding, so closures capture the correct value. `var` shares one binding.

8.4 Practical Uses of Closures

- Module pattern (private variables)
- Function factories
- Memoization
- Maintaining state in event handlers

8.5 Memory and Performance

- Closures keep referenced outer variables alive, which can lead to memory leaks if not handled.
 - In MCQs, they might ask about garbage collection: variables captured by a closure will not be garbage collected as long as the closure exists.
-

9. Asynchronous JavaScript

9.1 Synchronous vs Asynchronous

- **Synchronous:** blocking, one line at a time.
- **Asynchronous:** non-blocking, allows other code to run while waiting.

9.2 The Event Loop (Overview)

- Call stack checks if empty → processes microtasks (Promise callbacks) → then one macro-task (timer, I/O).
-

10. Callbacks

10.1 What is a Callback?

A function passed as an argument to another function, to be executed later.

```
function fetchData(callback) {  
  setTimeout(() => {  
    callback('Data received');  
  }, 1000);  
}  
fetchData((msg) => console.log(msg));
```

10.2 Callback Hell (Pyramid of Doom)

```
getUser(id, (user) => {
  getPosts(user.id, (posts) => {
    getComments(posts[0], (comments) => {
      // deeply nested
    });
  });
});
```

Problems: readability, error handling, inversion of control.

10.3 Error-First Callbacks (Node.js convention)

- First argument is an error object (if any).

```
fs.readFile('/path', (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});
```

MCQ Fact:

Always check for err first; ignoring it can cause silent failures.

10.4 Callback Problems Leading to Promises

- Callbacks are not composable.
 - Difficult to chain multiple async operations.
 - Error handling is scattered.
-

11. Promises

11.1 Definition

A **Promise** is an object representing the eventual completion (or failure) of an asynchronous operation. It has three states:

- **pending**
- **fulfilled** (resolved)
- **rejected**

11.2 Creating a Promise

```
const promise = new Promise((resolve, reject) => {
  // async work
  if (success) resolve(result);
  else reject(error);
});
```

11.3 Consuming Promises

- .then(onFulfilled, onRejected)
- .catch(onRejected)
- .finally(onFinally)

```
fetch('https://api.example.com')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(err => console.error(err))
  .finally(() => console.log('Done'));
```

11.4 Chaining

- Each .then() returns a new Promise, enabling chaining.
- Return a value, it becomes the resolution of the next .then().
- Return a Promise, the chain waits for it.

```
doFirst()
  .then(result => doSecond(result))
  .then(result2 => doThird(result2))
  .catch(error => handle(error)); // catches any error in chain
```

MCQ Fact:

If you forget to return from a .then(), the next .then() will receive undefined.

11.5 Promise Static Methods

Method	Purpose
Promise.resolve(val)	Returns a resolved promise.
Promise.reject(err)	Returns a rejected promise.
Promise.all(iterable)	Resolves when all promises resolve; rejects on first rejection.
Promise.race(iterable)	Settles as soon as any promise settles.

<code>Promise.allSettled()</code>	Resolves after all settle, with array of outcomes (status + value/reason).
<code>Promise.any()</code>	Resolves when first promise fulfills; rejects if all reject (AggregateError).

MCQ Trap:

`Promise.all` fails fast: if one promise rejects, the whole `.catch()` runs immediately. Other promises continue to run but their results are ignored.

`Promise.allSettled` is used when you need all results regardless of failure.

11.6 Microtasks

- Promise callbacks (`.then/.catch/.finally`) are **microtasks**.
- Microtasks execute after the current task finishes and before the next macro-task.
- `process.nextTick` in Node.js has even higher priority (microtask queue differs).

MCQ:

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
// Output: 1, 4, 3, 2
```

11.7 Promise vs Callback Advantages

- Better error propagation (catch at any point).
- Chainability and composability.
- Avoid inversion of control.

12. Async/Await

12.1 Syntax

- `async` functions always return a Promise.
- `await` pauses the execution until the Promise settles.

```

async function fetchData() {
  try {
    const response = await fetch(url);
    const data = await response.json();
    return data;
  } catch (error) {
    console.error(error);
    throw error; // re-throw to let caller handle
  }
}

```

12.2 Error Handling

- Use try...catch inside async functions.
- Or attach .catch() to the function call.

```

fetchData().then(data => {}).catch(err => {});

```

12.3 Sequential vs Parallel Execution

- await in loop runs sequentially, which may be slow.

```

for (const url of urls) {
  const res = await fetch(url); // one after another
}

```

- For parallel: Promise.all

```

const responses = await Promise.all(urls.map(url => fetch(url)));

```

MCQ Fact:

Using await inside a loop is not automatically parallel. Use Promise.all for concurrency.

12.4 Awaiting Non-Promises

- await 42 simply wraps it into Promise.resolve(42).
- The function pauses but resumes as a microtask.

```
async function test() {  
  console.log(1);  
  await 2; // equivalent to await Promise.resolve(2)  
  console.log(3);  
}  
test();  
console.log(4);  
// Output: 1, 4, 3
```

12.5 Async/Await vs Promises

- Syntax sugar over Promises.
- Makes asynchronous code look synchronous.
- Easier error handling with try/catch.

MCQ:

An async function always returns a Promise. Even if you return a primitive, it's wrapped. If the function throws, it returns a rejected Promise.

13. Node.js Runtime

13.1 What is Node.js?

- An open-source, cross-platform JavaScript runtime built on Chrome's V8 engine and the **libuv** library.
- Enables JavaScript to run outside the browser, with access to OS-level APIs.

13.2 Key Features

- Single-threaded event loop.
- Non-blocking I/O.
- Module system (CommonJS by default, ES modules supported).
- Rich built-in modules (fs, http, path, os, etc.).
- NPM ecosystem.

13.3 Global Objects in Node.js

- `global` (similar to `window` in browsers)
- `process` (info about the current process)
- `console`
- `Buffer`, `__dirname`, `__filename`, `module`, `exports`, `require` (CommonJS)
- In ES modules, `__dirname` and `__filename` are not available; use `import.meta.url`.

MCQ Fact:

var in the top-level of a CommonJS module does not leak to the global scope (module scope). But in browsers, top-level var becomes a property of window. In Node ES modules, top-level var is scoped to the module as well.

13.4 The process Object

- `process.argv`: command-line arguments.
- `process.env`: environment variables.
- `process.exit([code])`: exit the process.
- `process.nextTick(callback)`: defer execution until after current operation, before any I/O.
- `process.cwd()` vs `__dirname`: `cwd` is current working directory (from where node was launched), `__dirname` is the directory of the current module.

```
console.log(process.argv); // [node, script, ...args]
```

MCQ:

`process.nextTick` callbacks run before any other microtasks (like resolved promises) and before I/O callbacks. It can starve the event loop if used recursively.

14. Script Execution in Node.js

14.1 Running a Script

```
node script.js
```

- The file is wrapped in a Module Wrapper function (CommonJS):

```
(function(exports, require, module, __filename, __dirname) {  
  // module code  
});
```

This provides the local scope.

14.2 REPL

- Read-Eval-Print Loop: node without arguments.
- Each command has its own implicit scope.

14.3 Execution Order in a Node.js Application

1. Script initialization: top-level synchronous code.
 2. Event loop starts: timers, pending callbacks, idle/prepare, poll, check, close.
 3. Microtasks (nextTick, Promises) interleaved.
-

15. Module System (require / import)

15.1 CommonJS (require / module.exports)

- Default in Node.js (.js files).
- Synchronous, dynamic loading (can require conditionally).
- Each module has its own scope.
- Cached after first load.

```
// math.js
const add = (a, b) => a + b;
module.exports = { add }; // or exports.add = add;
// app.js
const math = require('./math');
console.log(math.add(2, 3));
```

MCQ Fact:

exports is a reference to module.exports. If you reassign exports = something, it breaks the link; module.exports remains empty. Use module.exports to export a single entity or override.

15.2 ES Modules (import / export)

- Use .mjs extension or "type": "module" in package.json.
- Static, asynchronous (dynamic import() returns a promise).
- Supports tree shaking.
- Top-level await allowed (in modules).

```
// math.mjs
export const add = (a, b) => a + b;
export default function multiply(a, b) { return a * b; }
// app.mjs
import multiply, { add } from './math.mjs';
console.log(add(1,2));
```

15.3 Key Differences

Feature	CommonJS	ES Modules
---------	----------	------------

Syntax	<code>require()</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Loading	Synchronous	Asynchronous
Strict Mode	Not by default (but modules are in strict mode)	Always strict
this at top level	<code>exports</code> / <code>module.exports</code> (or empty object)	undefined
Dynamic loading	<code>require()</code> anywhere	<code>import()</code> function (promise)
Caching	Yes	Yes
<code>__dirname</code> / <code>__filename</code>	Available	Not available (use <code>import.meta.url</code>)

MCQ Fact:

ES modules are statically analyzable, enabling better tooling like tree shaking.
CommonJS is dynamic.

15.4 Module Caching

- Modules are cached after the first `require()`. Subsequent `require()` calls return the same object (singleton).
- To avoid caching: delete from `require.cache` or use different technique.
- This can be a source of stale configuration if you mutate the exported object.

```
delete require.cache[require.resolve('./config')];
```

16. File System (fs)

16.1 Import

```
const fs = require('fs');           // CommonJS
import fs from 'fs';                // ES (or import * as fs)
const fsp = require('fs/promises'); // Promise-based
```

16.2 Synchronous vs Asynchronous vs Promise API

- `fs.readFileSync(path)`: blocking, returns data.
- `fs.readFile(path, callback)`: non-blocking, error-first callback.
- `fs.promises.readFile(path)`: returns Promise, supports async/await.

MCQ Fact:

In server applications, **never** use synchronous methods like `readFileSync` inside request handlers; they block the event loop, hurting concurrency.

16.3 Common Methods

Method	Description
<code>readFile</code>	Read entire file
<code>writeFile</code>	Write (overwrites) file
<code>appendFile</code>	Append data to file
<code>unlink</code>	Delete file
<code>mkdir</code>	Create directory
<code>readdir</code>	Read directory contents
<code>stat</code>	Get file information (size, isFile, etc.)
<code>rename</code>	Move/rename file
<code>watch</code>	Watch for file changes

16.4 Example: Read with callbacks and promises

```
// Callback
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});

// Promise
fs.promises.readFile('file.txt', 'utf8')
  .then(data => console.log(data))
  .catch(err => console.error(err));

// Async/await
try {
  const data = await fs.promises.readFile('file.txt', 'utf8');
  console.log(data);
} catch (err) {
  console.error(err);
}
```

16.5 Working with JSON Files

```
const config = JSON.parse(fs.readFileSync('config.json', 'utf8'));
// or async
const data = await fs.promises.readFile('data.json', 'utf8');
const obj = JSON.parse(data);
```

Watch out: `JSON.parse` can throw on invalid JSON. Always handle errors.

16.6 File Descriptors and Flags

- `r`, `r+`, `w`, `w+`, `a`, `a+`.
- `fs.open` gives a file descriptor, which can be used with `fs.read` etc.

MCQ:

'w' flag truncates file if exists; 'a' appends. 'r+' requires file exists and opens for reading and writing.

17. JSON Handling

17.1 `JSON.stringify`

- Converts a JavaScript value to JSON string.
- Skips undefined, Function, Symbol values (in objects).
- Throws on circular references.
- **Replacer** (second argument) can filter/transform values.
- **Space** (third argument) for indentation.

```
const obj = { a: 1, b: undefined, c: () => {} };
JSON.stringify(obj); // '{"a":1}'
JSON.stringify(obj, null, 2); // pretty print
```

17.2 `JSON.parse`

- Parses JSON string into JavaScript value.
- **Reviver** function allows transformation during parsing.

```
const str = '{"date":"2023-01-01"}';
const obj = JSON.parse(str, (key, value) => {
  if (key === 'date') return new Date(value);
  return value;
});
console.log(obj.date instanceof Date); // true
```

17.3 Deep Clone using JSON

```
const deepClone = obj => JSON.parse(JSON.stringify(obj));
```

MCQ Trap:

This method loses:

- Functions
- Undefined
- Symbols
- Circular references (error)
- Special objects like Date become strings.

So it's not a reliable deep clone for complex objects. Use `structuredClone()` or libraries.

17.4 JSON vs JavaScript Object Literal

- JSON requires double quotes for strings and keys; trailing commas not allowed.
 - JSON supports only: string, number, object, array, boolean, null.
-

18. Environment Variables

18.1 `process.env`

- An object containing the user environment variables.
- All values are **strings**.

```
const PORT = process.env.PORT || 3000;  
console.log(process.env.NODE_ENV);
```

18.2 Using `.env` files with `dotenv`

```
npm install dotenv
```

```
require('dotenv').config(); // Loads .env into process.env
```

File `.env`:

```
PORT=8080
DB_URL=mongodb://localhost/test
```

MCQ Fact:

Never commit .env files to version control; they may contain secrets.

18.3 Cross-Platform Considerations

- On Windows, `process.env.VARNAME` is case-insensitive.
 - On Unix, it's case-sensitive. Prefer `UPPER_CASE` naming.
-

19. Event Loop (Deep Dive)

19.1 Phases of the Node.js Event Loop

1. **Timers:** callbacks scheduled by `setTimeout()`, `setInterval()`.
2. **Pending callbacks:** I/O callbacks deferred to next loop iteration.
3. **Idle, prepare:** internal.
4. **Poll:** retrieve new I/O events; execute I/O related callbacks.
5. **Check:** `setImmediate()` callbacks.
6. **Close callbacks:** `socket.on('close', ...)`.

19.2 `process.nextTick` and Microtasks

- `process.nextTick` fires immediately after the current operation, before any other I/O events or timers.
- Promise callbacks (microtasks) run after `nextTick` callbacks.
- Within each phase, before moving to the next phase, `nextTick` and Promise microtasks are flushed.

MCQ Order:

```
setImmediate(() => console.log('immediate'));
setTimeout(() => console.log('timeout'), 0);
Promise.resolve().then(() => console.log('promise'));
process.nextTick(() => console.log('nextTick'));
console.log('start');
// Output: start, nextTick, promise, timeout, immediate? Or immediate?
// Depends on context: in main module, timers vs check order can be
// Typically: setTimeout(fn,0) and setImmediate order is not guaranteed
```

Accurate Fact:

- Outside I/O cycle, the order between `setTimeout(...,0)` and `setImmediate` is non-deterministic (depends on system scheduling).
- Inside an I/O callback (e.g., `fs.readFile`), `setImmediate` always runs before `setTimeout`.

```
fs.readFile(__filename, () => {
  setTimeout(() => console.log('timeout'), 0);
  setImmediate(() => console.log('immediate'));
  // Output: immediate, timeout
});
```

MCQ Fact:

`process.nextTick` always runs before Promise callbacks if both are scheduled in the same tick.

19.3 Microtask vs Macrotask

- **Microtasks:** `process.nextTick`, `Promise.then/catch/finally`, `queueMicrotask`.
- **Macrotasks:** `setTimeout`, `setInterval`, `setImmediate`, I/O callbacks.

Event loop picks **one macrotask**, then processes **all microtasks** before next macrotask.

19.4 Starvation

- Continuously adding `nextTick` or microtasks can starve the event loop, preventing I/O and timers from executing.

20. EventEmitter

20.1 Definition

- Node.js core class `events.EventEmitter` enables event-driven architecture.
- Objects can emit named events, and listeners can subscribe to them.

20.2 Basic Usage

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

emitter.on('event', (arg) => {
  console.log('listener', arg);
});
emitter.emit('event', { data: 123 });
```

20.3 Methods

Method	Description
on(event, listener)	Add listener (alias addListener)
once(event, listener)	Add one-time listener (removed after first emit)
off(event, listener)	Remove listener (alias removeListener)
emit(event, ...args)	Emit event, returns true if listeners existed
listenerCount(event)	Number of listeners for event
removeAllListeners([event])	Remove all or specific event listeners
prependListener(event, listener)	Add to front of listeners array

20.4 Error Events

- If an error event is emitted and no listener is attached, the error is thrown, crashing the process.
- Always listen for error.

```
emitter.on('error', (err) => {
  console.error('Error caught:', err);
});
```

20.5 Inheriting EventEmitter

Often you extend EventEmitter to give custom classes event capabilities.

```
class MyEmitter extends EventEmitter {}
const myEmitter = new MyEmitter();
```


20.6 Max Listeners

- Default max listeners per event: 10.
- Exceeding prints a warning. Increase with `emitter.setMaxListeners(n)`.

MCQ:

Listener functions are called synchronously in the order they were added. Avoid blocking the event loop inside listeners.

21. Streams

21.1 What are Streams?

- Streams are collections of data that might not be available all at once, and don't have to fit in memory.
- They process data chunk by chunk, making them efficient for large data.

21.2 Types of Streams

1. **Readable** – from which data can be read (e.g., `fs.createReadStream`).
2. **Writable** – to which data can be written (e.g., `fs.createWriteStream`).
3. **Duplex** – both Readable and Writable (e.g., `socket`).
4. **Transform** – Duplex that can modify data as it passes through (e.g., `compression`, `zlib.createGzip`).

21.3 Readable Stream Example

```
const readStream = fs.createReadStream('input.txt', { encoding: 'utf-8' });
readStream.on('data', (chunk) => {
  console.log('Chunk:', chunk);
});
readStream.on('end', () => console.log('No more data'));
readStream.on('error', (err) => console.error(err));
```

21.4 Writable Stream Example

```
const writeStream = fs.createWriteStream('output.txt');
writeStream.write('Hello ');
writeStream.write('World!');
writeStream.end(); // close the stream
```

21.5 Pipe

- `readable.pipe(writable)` connects streams, handling backpressure automatically.

```
readStream.pipe(writeStream);
```

- You can chain:

```
readStream
  .pipe(transformStream)
  .pipe(writeStream);
```

21.6 Transform Stream

```
const { Transform } = require('stream');
const upperCaseTransform = new Transform({
  transform(chunk, encoding, callback) {
    this.push(chunk.toString().toUpperCase());
    callback();
  }
});
process.stdin.pipe(upperCaseTransform).pipe(process.stdout);
```

21.7 Backpressure

- If writable is slower than readable, data buffers inside memory. Streams handle this through `highWaterMark` and `drain` event.
- `write()` returns `false` when internal buffer is full. Wait for `drain` event before writing more.

MCQ Fact:

Using `.pipe()` automatically manages backpressure. Manual piping without `drain` can cause memory overflow.

21.8 Object Mode

- Streams can work with objects instead of buffers/strings by setting `objectMode: true`.

```
const objectStream = new Transform({
  objectMode: true,
  transform(obj, enc, cb) {
    obj.processed = true;
    cb(null, obj);
  }
});
```

21.9 Stream Events

- **Readable:** data, end, error, close, readable (when data available to read manually).
- **Writable:** drain, finish, error, close, pipe/unpipe.

MCQ:

finish event on writable is emitted after end() is called and all data flushed. end on readable is when no more data.

21.10 pipeline and finished utilities (stream/promises)

- stream.pipeline(sources..., callback) – pipe and handle errors/cleanup automatically.
- stream.finished(stream, callback) – notified when stream ends or errors.

```
const { pipeline } = require('stream/promises');
await pipeline(
  fs.createReadStream('in'),
  zlib.createGzip(),
  fs.createWriteStream('out.gz')
);
```

MCQ Fact:

pipeline ensures proper cleanup and error propagation, better than manual .pipe() chains.

22. Additional Important Concepts for MCQs

22.1 this Binding Summary

- Global: window / global (undefined in strict mode).
- Method call: the object.
- Function call: undefined (strict) or global.
- Arrow: lexical.

- Constructor: new instance.
- call/apply/bind: explicitly set.
- Event handlers: element (DOM), or emitter (Node EventEmitter).

22.2 bind, call, apply

- call(thisArg, arg1, arg2...) – invoke immediately.
- apply(thisArg, [args]) – invoke immediately.
- bind(thisArg, ...args) – returns new function with bound this and partial arguments.

```
function greet(greeting) { return `${greeting}, ${this.name}`; }
const person = { name: 'Alice' };
console.log(greet.call(person, 'Hello')); // Hello, Alice
const bound = greet.bind(person);
console.log(bound('Hi')); // Hi, Alice
```

22.3 Prototype Chain

- Each object has a hidden `[[Prototype]]` property (accessible via `__proto__` or `Object.getPrototypeOf`).
- Function objects have prototype property used when constructor is called with `new`.
- `Object.create(proto)` creates object with specified prototype.

MCQ:

`Object.prototype` is the top of the chain for most objects. `null` prototype objects have no inherited methods.

22.4 Strict Mode

- "use strict"; at top of file/function.
- Eliminates some silent errors, makes this undefined in functions, prevents duplicate parameters.
- Modules (ES) and classes are always strict.

22.5 Error Types and Handling

- Error, SyntaxError, TypeError, ReferenceError, RangeError.
- try...catch...finally for synchronous errors.
- For promises: `.catch()` or try/catch with `await`.
- Unhandled promise rejections lead to Node process termination (warning in older versions).

22.6 setImmediate vs setTimeout(fn, 0) vs process.nextTick

Function	Phase	Usage
<code>process.nextTick</code>	microtask	Execute before next event loop tick
<code>setImmediate</code>	check phase	Execute after I/O events in current iteration
<code>setTimeout</code>	timers phase	Delay by at least ms (min 0)

MCQ:

`process.nextTick` starves I/O if called recursively; use `setImmediate` to give I/O a chance.

22.7 Memory Leaks

- Global variables.
- Timers not cleared.
- Event listeners not removed.
- Closures holding large objects.

22.8 require vs import for JSON

- CommonJS: `const data = require('./file.json')` works directly (parsed automatically).
- ES modules: you must use `fs` or experimental `import assertions`: `import data from './file.json' assert { type: 'json' }.`

23. Practice MCQ Collection (Examples)

Here are sample MCQ-style questions to test understanding:

Q1: What will be the output?

```
console.log(typeof null);
```

- a) "null"
- b) "object"
- c) "undefined"
- d) "null"

Answer: b) "object"

Q2: Which method is used to stop the event loop from proceeding to the next phase until the current operation completes? a) `setImmediate` b) `process.nextTick` c) `setTimeout` d) None, event loop always completes phases.

Q3: What is the correct way to deep clone an object that may contain Dates and Functions? a) `JSON.parse(JSON.stringify(obj))` b) `structuredClone(obj)` c) `Object.assign({}, obj)` d) Spread operator **Answer:** b) `structuredClone` (supports more types; part of HTML spec, available in Node 17+).

Q4: In Node.js, which statement about ES modules is true? a) `__dirname` is available inside ES modules. b) ES modules are synchronous. c) `import` can be used conditionally inside an if statement. d) Top-level `this` is undefined. **Answer:** d) Top-level `this` is undefined.

Q5: What will happen if an 'error' event is emitted on an EventEmitter that has no 'error' listener? a) The event is ignored. b) The error is written to stderr. c) The process exits. d) A warning is printed and execution continues. **Answer:** c) The error is thrown and the process crashes.